

## Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 #define MAX 100
6
7 /* -----
8    1. SCHEDULING PROBLEM
9    Weighted Interval Scheduling using Dynamic Programming
10   ----- */
11
12 typedef struct {
13     int start;
14     int finish;
15     int profit;
16 } Job;
17
18 /* Sort jobs according to finish time */
19 void sortJobs(Job jobs[], int n) {
20     int i, j;
21     Job temp;
22
23     for (i = 0; i < n - 1; i++) {
24         for (j = 0; j < n - i - 1; j++) {
25             if (jobs[j].finish > jobs[j + 1].finish) {
26                 temp = jobs[j];
27                 jobs[j] = jobs[j + 1];
28                 jobs[j + 1] = temp;
29             }
30         }
31     }
32 }
33
```

## Input

stdin input

## Output

```
=====
DYNAMIC PROGRAMMING APPLICATIONS
Scheduling and String
Segmentation
=====
```

```

35 int findPrevious(Job jobs[], int i) {
36     int j;
37
38     for (j = i - 1; j >= 0; j--) {
39         if (jobs[j].finish <= jobs[i].start)
40             return j;
41     }
42
43     return -1;
44 }
45
46 void schedulingDP() {
47     Job jobs[] = {
48         {1, 3, 50},
49         {2, 5, 20},
50         {4, 6, 70},
51         {6, 7, 60},
52         {5, 8, 30},
53         {7, 9, 40}
54     };
55
56     int n = sizeof(jobs) / sizeof(jobs[0]);
57     int dp[MAX];
58     int prev[MAX];
59     int i, p;
60     int selected[MAX];
61     int count = 0;
62
63     sortJobs(jobs, n);
64
65     printf("\n===== SCHEDULING PROBLEM =====\n");
66
67     printf("\nJobs after sorting by finish time:\n");
68     printf("Job\tStart\tFinish\tProfit\n");
69

```

```
70     for (i = 0; i < n; i++) {
71         printf("%d\t%d\t%d\t%d\n",
72             i + 1,
73             jobs[i].start,
74             jobs[i].finish,
75             jobs[i].profit);
76     }
77
78     dp[0] = jobs[0].profit;
79     prev[0] = -1;
80
81     for (i = 1; i < n; i++) {
82
83         p = findPrevious(jobs, i);
84
85         int include = jobs[i].profit;
86
87         if (p != -1)
88             include += dp[p];
89
90         int exclude = dp[i - 1];
91
92         if (include > exclude) {
93             dp[i] = include;
94             prev[i] = p;
95         } else {
96             dp[i] = exclude;
97             prev[i] = -2;
98         }
99     }
100
101     printf("\nDP Table:\n");
102     for (i = 0; i < n; i++)
103         printf("dp[%d] = %d\n", i, dp[i]);
104
```

```
106     i = n - 1;
107
108     while (i >= 0) {
109
110         p = findPrevious(jobs, i);
111
112         int include = jobs[i].profit;
113
114         if (p != -1)
115             include += dp[p];
116
117         if (include >= dp[i]) {
118             selected[count++] = i;
119             i = p;
120         } else {
121             i--;
122         }
123     }
124
125     printf("\nSelected Jobs:\n");
126
127     for (i = count - 1; i >= 0; i--) {
128         int j = selected[i];
129
130         printf("Job %d: Start = %d, Finish = %d, Profit = %d\n",
131             j + 1,
132             jobs[j].start,
133             jobs[j].finish,
134             jobs[j].profit);
135     }
136
137     printf("\nMaximum Profit = %d\n", dp[n - 1]);
138
139     printf("\nRecurrence Relation:\n");
140     printf("DP[i] = max(DP[i-1], Profit[i] + DP[p(i)])\n");
```

```

142     printf("Decision: Include the current job OR exclude the current
143
144     printf("\nTime Complexity: O(n^2) with the current previous-job s
145     printf("Space Complexity: O(n)\n");
146 }
147
148
149 /* -----
150 2. STRING SEGMENTATION PROBLEM
151 Word Break using Dynamic Programming
152 ----- */
153
154 int dictionaryContains(char word[]) {
155
156     char dictionary[][20] = {
157         "i",
158         "like",
159         "sam",
160         "sung",
161         "samsung",
162         "mobile",
163         "ice",
164         "cream",
165         "icecream"
166     };
167
168     int size = sizeof(dictionary) / sizeof(dictionary[0]);
169     int i;
170
171     for (i = 0; i < size; i++) {
172         if (strcmp(word, dictionary[i]) == 0)
173             return 1;
174     }
175
176     return 0;

```

```

177 }
178
179 void stringSegmentationDP() {
180
181     char str[] = "ilikesamsung";
182     int n = strlen(str);
183
184     int dp[MAX];
185     int parent[MAX];
186
187     int i, j;
188     char word[MAX];
189
190     for (i = 0; i <= n; i++) {
191         dp[i] = 0;
192         parent[i] = -1;
193     }
194
195     dp[0] = 1;
196
197     for (i = 1; i <= n; i++) {
198
199         for (j = 0; j < i; j++) {
200
201             int length = i - j;
202
203             strncpy(word, str + j, length);
204             word[length] = '\0';
205
206             if (dp[j] && dictionaryContains(word)) {
207                 dp[i] = 1;
208                 parent[i] = j;
209                 break;
210             }
211         }

```

```

211     }
212 }
213
214 printf("\n\n===== STRING SEGMENTATION =====\n");
215
216 printf("\nInput String: %s\n", str);
217
218 printf("\nDP Table:\n");
219
220 for (i = 0; i <= n; i++)
221     printf("dp[%d] = %d\n", i, dp[i]);
222
223 printf("\nRecurrence Relation:\n");
224 printf("DP[i] = OR(DP[j] AND Dictionary[j...i-1])\n");
225
226 printf("Decision: Choose a valid dictionary word ending at position\n");
227
228 if (dp[n]) {
229
230     printf("\nString can be segmented.\n");
231
232     printf("Segmentation: ");
233
234     int positions[MAX];
235     int count = 0;
236
237     i = n;
238
239     while (i > 0) {
240         positions[count++] = i;
241         i = parent[i];
242     }
243
244     positions[count++] = 0;
245

```

```

246         for (i = count - 1; i > 0; i--) {
247
248             int start = positions[i];
249             int end = positions[i - 1];
250
251             printf("%.s", end - start, str + start);
252
253             if (i > 1)
254                 printf(" | ");
255         }
256
257         printf("\n");
258
259     } else {
260
261         printf("\nString cannot be segmented.\n");
262     }
263
264     printf("\nTime Complexity:  $O(n^2 * L)$ \n");
265     printf("Space Complexity:  $O(n)$ \n");
266 }
267
268
269 /* -----
270 MAIN FUNCTION
271 ----- */
272
273 int main() {
274
275     printf("=====\n");
276     printf(" DYNAMIC PROGRAMMING APPLICATIONS\n");
277     printf(" Scheduling and String Segmentation\n");
278     printf("=====\n");
279
280     schedulingDP();
281

```



```

282     stringSegmentationDP();
283
284     printf("\n\n===== COMPARISON =====\n");
285
286     printf("\nScheduling:\n");
287     printf("State: Maximum profit up to job i\n");
288     printf("Decision: Include or exclude current job\n");
289     printf("Recurrence: DP[i] = max(DP[i-1], profit[i] + DP[p(i)])\n");
290     printf("Constraint: Jobs must not overlap\n");
291
292     printf("\nString Segmentation:\n");
293     printf("State: Whether prefix of length i can be segmented\n");
294     printf("Decision: Select a valid dictionary word\n");
295     printf("Recurrence: DP[i] = OR(DP[j] AND validWord(j,i))\n");
296     printf("Constraint: Substrings must exist in dictionary\n");
297
298     printf("\n===== \n");
299     printf(" DP demonstrates versatility by solving both\n");
300     printf(" optimization and decision problems.\n");
301     printf("===== \n");
302
303     return 0;
304 }

```

✓ Submitted